

Lab 05 - Container Images, OCI and the Runtime Interface

Lab id	Lab 05
Day	1 - Cloud Native Foundations & Kubernetes Core Concepts
Duration	40 minutes
Namespace	kcna-lab05
Mapping	LO1 Develop a Kubernetes architectural proof of concept A1 Develop an architectural proof of concept K1 Process for developing proof of concepts

1. Objective

By the end of this lab you will be able to:

- Decompose an image reference into registry, repository, tag and digest, and state the two defaults Kubernetes applies silently.
- Describe the OCI Image Specification stack - index → manifest → config → content-addressed layers - and why layers are shared between images.
- Distinguish `spec.containers[].image` (what you asked for) from `status.containerStatuses[].imageID` (what the runtime resolved).
- State the CRI boundary: kubelet → CRI → containerd/CRI-O → OCI runtime.
- Apply `imagePullPolicy` correctly, including its version-dependent default, and recognise `ErrImageNeverPull`.
- Explain and demonstrate why a moved tag breaks reproducibility and a digest cannot.

2. Prerequisites

- Kubernetes v1.30+ cluster; `kubectl` on PATH.
- Labs 01-04 completed.
- Outbound access to Docker Hub from the cluster node (the lab pulls `nginx:1.27-alpine` and `busybox:1.36`).

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	102m	v1.37.0

```
cd courseware/labs/lab-05-container-images-oci
```

3. Scenario

Meridian Freight Pte Ltd had an outage last quarter that nobody could reproduce. The Depot Portal ran fine in staging and failed in production, from the **same** manifest, with the **same** image reference: nginx:1.27-alpine.

The post-incident review found the cause. The two environments had pulled that tag six weeks apart, and in between the publisher had re-pointed it at a new build. Two nodes, one reference, two different images.

The architecture board has asked the PoC to answer three questions:

1. What exactly does an image reference name?
2. How do we prove which bytes are running?
3. What is the minimum change that makes deployments reproducible?

Meridian's approved image list is data/image-inventory.csv, and the reference normalisation card the team keeps on the wall is data/reference-forms.tsv. You will serve both from the cluster and use them to answer all three questions.

4. Step-by-step procedure

Step 1 - Namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab05 created
```

Step 2 - Publish the inventory datasets

```
kubectl -n kcna-lab05 create configmap image-catalog \
  --from-file=data/image-inventory.csv \
  --from-file=data/reference-forms.tsv
```

```
configmap/image-catalog created
```

```
kubectl -n kcna-lab05 get configmap image-catalog \
  -o go-template='{range $k,$v := .data}}{{ $k }}{{ "\n" }}{{ end }}'
```

```
image-inventory.csv
reference-forms.tsv
```

Step 3 - What is actually running containers on this node

Kubernetes does not run containers. The kubelet speaks the Container Runtime Interface (CRI) - a gRPC API - to a runtime, and that runtime speaks the OCI Runtime Spec to a low-level runtime that creates the process.

```
kubelet ■■■CRI (gRPC)■■■■ containerd / CRI-0 ■■■OCI runtime spec■■■■ runc / crun
                                   ■■■ OCI distribution spec ■■■ registry (Docker Hub, registry.k8s.io, ...)
```

Ask the node which runtime it has:

```
kubectl get nodes -o jsonpath='{.items[0].status.nodeInfo.containerRuntimeVersion}{"\n"}'
```

```
containerd://2.1.4
```

Everything before `://` is the CRI implementation. Since Kubernetes v1.24 there is no built-in Docker shim: Docker Engine is not a CRI runtime, though images built by Docker are ordinary OCI images and run fine.

On a node you could inspect the same thing with `crictl` images,

`crictl inspecti <ref>` and `crictl ps - crictl` is the CRI-level equivalent

of docker. It needs shell access to the node, which this lab deliberately

does not require. Every fact `crictl` would show you is available through the

API, as the next command demonstrates.

The kubelet reports the node's whole image store back to the API server:

```
kubectl get nodes \
  -o jsonpath='{range .items[0].status.images[*]}{.sizeBytes}{"\t"}{.names[0]}{"\n"}{end} ' \
  | sort -rn | head -n 6
```

```
147718971■registry.k8s.io/etcd@sha256:
e78e30bb5c2ea9d2b0f8b1a2f9e1bd5e9e9b3f18ea9d1c0b2b74f6c3e2e9b40d2
92632544■registry.k8s.io/kube-apiserver@sha256:
1a1f05a2cd7c2b2e3e0f1a45c7c7f74a4e3fdc7fbc25da8b0e0d84a8b7f6cf12
21674089■docker.io/library/nginx@sha256:
3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
4264178■docker.io/library/alpine@sha256:
beefdbd8a1da6d2915566fde36db9db0b524eb737fc57cd1367effd16dc0d06d
4042190■docker.io/library/busybox@sha256:
2376a0c12759aa1214ba83e771ff252c7b1663216b192f5e0fb364e952f85c
744000■registry.k8s.io/pause@sha256:
7031c1b283388d2c2e09b57badb803c05ebd362dc88d84b480cc47f72a21097
```

Two things to take away:

- Every entry is addressed by digest, not by tag. The node's store is a content-addressed cache; tags are labels pointing into it.
- `registry.k8s.io/pause` is the sandbox (infrastructure) container. Every

Pod gets one; it holds the network namespace open so that your containers - and the sidecars from Lab 04 - can join it and share a Pod IP.

Step 4 - Anatomy of an image reference

```
grep -v '^#' data/reference-forms.tsv | column -t -s '$\t'
```

nginx	docker.io/library/nginx:latest	weak -
tag is mutable AND implicit		
nginx:1.27-alpine	docker.io/library/nginx:1.27-alpine	weak -
tag can be re-pointed		
library/nginx:1.27-alpine	docker.io/library/nginx:1.27-alpine	weak -
same as above		
docker.io/library/nginx:1.27-alpine	docker.io/library/nginx:1.27-alpine	weak -
fully qualified but still a tag		
nginx@sha256:<64 hex>	docker.io/library/nginx@sha256:<64 hex>	strong -
content addressed, immutable		
registry.k8s.io/pause:3.9	registry.k8s.io/pause:3.9	weak - no
docker.io default applied		
ghcr.io/meridian/depot-api:2.4.1	ghcr.io/meridian/depot-api:2.4.1	weak -
private registry, still a tag		
localhost:5000/depot-api:dev	localhost:5000/depot-api:dev	weak -
host:port prefix means "registry", not "repo"		

A full reference is:

```
[REGISTRY[:PORT]/]REPOSITORY[:TAG][@DIGEST]
  docker.io / library/nginx : 1.27-alpine @ sha256:3d69...
```

Two defaults are applied silently when you write a short name:

1. No registry host → docker.io, and for a single-segment repository also

library/. This is a Docker Hub convention that other registries do not share: registry.k8s.io/pause gets no library/.

2. No tag and no digest → :latest. latest is not "the newest"; it is

just the default tag name, and it may be years old or may not exist at all.

The rule of thumb Kubernetes decides on: a reference is *anything* the runtime can resolve. How much you can trust it is entirely up to you.

Step 5 - Run the tag-pinned catalog and read what it resolved

```
kubectl apply -f manifests/10-image-catalog.yaml
```

```
pod/image-catalog created
```

```
kubectl -n kcna-lab05 wait --for=condition=Ready pod/image-catalog --timeout=180s
```

```
pod/image-catalog condition met
```

Now read the three fields that matter:

```
kubectl -n kcna-lab05 get pod image-catalog \
-o jsonpath='{.spec.containers[0].image}{"\n"}{.status.containerStatuses[0].image}{"\n"}{.status.containerStatuses[0].imageID}{"\n"}'
```

```
nginx:1.27-alpine
docker.io/library/nginx:1.27-alpine
docker.io/library/nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

Field	Meaning
spec.containers[0].image	what you wrote - never rewritten by Kubernetes
status.containerStatuses[0].image	the normalised reference the runtime used
status.containerStatuses[0].imageID	the digest that actually ran

That third line is your answer to "which bytes are in production?". Record it at every release. Save it now - you will need it in section 6:

```
CATALOG_ID=$(kubectl -n kcna-lab05 get pod image-catalog \
-o jsonpath='{.status.containerStatuses[0].imageID}')
echo "$CATALOG_ID"
```

```
docker.io/library/nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

Confirm the datasets are being served:

```
kubectl -n kcna-lab05 exec image-catalog -c web -- \
  wget -qO- http://127.0.0.1/image-inventory.csv | grep -v '^#' | head -n 3
```

```
component,image_ref,registry,repository,tag,pin_style,approx_size_mb,base,notes
depot-web,nginx:1.27-alpine,docker.io,library/nginx,1.27-alpine,tag,22,alpine,static content
only
depot-api,busybox:1.36,docker.io,library/busybox,1.36,tag,2,scratch+busybox,PoC stand-in for
the Go API
```

Step 6 - What is inside an OCI image

An OCI image is not a file. It is a small graph of JSON documents plus a set of compressed tarballs, all addressed by the SHA-256 of their own content:

```
image index      (optional, multi-platform: "which manifest for linux/arm64?")
  ■■■ image manifest (one platform: lists the config + the layers)
    ■■■ image config (JSON: ENTRYPOINT, CMD, Env, WorkingDir, User, rootfs.diff_ids,
history)
      ■■■ layers[]   (gzipped tar of filesystem CHANGES, each with its own digest)
```

Consequences you can reason about without a shell on the node:

- Layers are shared. nginx:1.27-alpine and alpine:3.20 have the same

Alpine base layer, stored once. The node's image store above lists the *image* sizes, not the sum of unique bytes on disk.

- The image config is where ENTRYPOINT and CMD live - the exact fields you

override with command and args in Lab 03. kubectl never shows them, because Kubernetes does not read them; the runtime does.

- Multi-platform is an index, not a different image. The same

busybox:1.36 reference resolves to a different manifest on arm64 and amd64. That is why a digest you copied from an amd64 laptop can fail on an arm64 node: you pinned the *platform manifest* instead of the *index*.

- A digest is a checksum of the manifest. If any byte of any layer changed,

the layer digest changes, so the manifest changes, so the image digest changes. There is no way to alter content behind a digest.

Look at the immutable identity Kubernetes itself keeps for the sandbox image:

```
kubectl get nodes \
-o jsonpath='{range .items[0].status.images[*]}.{.names[0]}{"\n"}{end}' | grep pause
```

```
registry.k8s.io/pause@sha256:7031c1b283388d2c2e09b57badb803c05ebed362dc88d84b480cc47f72a21097
```

Step 7 - imagePullPolicy semantics

```
kubectl apply -f manifests/30-pullpolicy-matrix.yaml
```

```
pod/pp-always created
pod/pp-ifnotpresent created
pod/pp-never created
```

```
kubectl -n kcna-lab05 get pods -o custom-columns=\
'NAME:.metadata.name,IMAGE:.spec.containers[0].image,POLICY:.spec.containers[0].
imagePullPolicy,STATUS:.status.phase'
```

NAME STATUS	IMAGE	POLICY
image-catalog Running	nginx:1.27-alpine	IfNotPresent
pp-always Running	busybox:1.36	Always
pp-ifnotpresent Running	busybox:1.36	IfNotPresent
pp-never Pending	meridianfreight.example.com/depot-portal/depot-api:0.1.0-poc	Never

Compare the events:

```
kubectl -n kcna-lab05 describe pod pp-always | grep -E 'Pulling|Pulled'
```

```
Normal Pulling 30s kubelet Pulling image "busybox:1.36"
Normal Pulled 28s kubelet Successfully pulled image "busybox:1.36" in 1.842s (1.842s
including waiting). Image size: 4042190 bytes.
```

```
kubectl -n kcna-lab05 describe pod pp-ifnotpresent | grep -E 'Pulling|Pulled'
```

```
Normal Pulled 30s kubelet Container image "busybox:1.36" already present on machine
```

No Pulling line at all for IfNotPresent - the node had it. And the third:

```
kubectl -n kcna-lab05 get pod pp-never
```

```
NAME          READY   STATUS             RESTARTS   AGE
pp-never      0/1     ErrImageNeverPull   0           19s
```

```
kubectl -n kcna-lab05 describe pod pp-never | tail -n 4
```

```
Events:
  Type    Reason             Age   From    Message
  ----    -
Warning  ErrImageNeverPull  6s (x3 over 21s)  kubelet  Container image "meridianfreight.example.com/depot-portal/depot-api:0.1.0-poc" is not present with pull policy of Never
```

Note the distinction from Lab 02: this is ErrImageNeverPull, not ErrImagePull. No registry was contacted; there is no network failure to investigate. Never is used with pre-loaded images (kind load docker-image) and in air-gapped clusters.

Policy	Behaviour	Use when
Always	resolve the reference at the registry on every container start	mutable tags, or you must guarantee freshness; fails if the registry is down
IfNotPresent	use the node's copy if the reference is present	fixed tags and digests - the normal choice
Never	never contact a registry	air-gapped nodes, pre-loaded images

Defaults, if you omit the field: :latest or no tag at all → Always; any other explicit tag → IfNotPresent; a digest → IfNotPresent. State the policy explicitly anyway, so a reviewer never has to recall this table.

5. Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script checks the namespace and both ConfigMap keys; that the node reports a CRI runtime; that image-catalog is Ready, that its spec.image is the tag you wrote and its status.imageID is a well-formed sha256:<64 hex> digest; that both datasets are actually served over HTTP; that the running catalog image is on data/image-inventory.csv while pp-never's image deliberately is not; the three imagePullPolicy values and pp-never's ErrImageNeverPull; and finally that depot-web-locked is digest-pinned and - once you have repaired it - that its digest matches image-catalog's exactly.

Expected tail:

```
Result: 19 passed, 0 failed
```

Full annotated evidence: [`verification/expected-output.md`](#).

6. Failure injection - the tag moved, or the digest went stale

manifests/20-digest-pinned.yaml carries a deliberately stale digest: a syntactically valid sha256: reference that no registry can resolve. This is exactly what a hand-copied digest looks like after the manifest it named was replaced or the operator mistyped one character.

```
kubectl apply -f manifests/20-digest-pinned.yaml
```

```
pod/depot-web-locked created
```

```
kubectl -n kcna-lab05 get pod depot-web-locked
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-locked	0/1	ErrImagePull	0	7s

```
kubectl -n kcna-lab05 describe pod depot-web-locked | tail -n 8
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	48s	default-scheduler	Successfully assigned kcna-lab05/depot-web-locked to kcna-qa-20260905-control-plane
Normal	Pulling	18s (x3 over 48s)	kubelet	Pulling image "nginx@sha256:9c1b5b6f0f6cc1e7d0d2a3c4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718"
Warning	Failed	17s (x3 over 47s)	kubelet	Failed to pull image "nginx@sha256:9c1b5b6f...": failed to pull and unpack image "docker.io/library/nginx@sha256:9c1b5b6f...": failed to resolve reference "docker.io/library/nginx@sha256:9c1b5b6f...": docker.io/library/nginx@sha256:9c1b5b6f...: not found
Warning	Failed	17s (x3 over 47s)	kubelet	Error: ErrImagePull
Normal	BackOff	3s (x4 over 46s)	kubelet	Back-off pulling image "nginx@sha256:9c1b5b6f..."
Warning	Failed	3s (x4 over 46s)	kubelet	Error: ImagePullBackOff

Diagnosis

- The reference is syntactically valid, so nothing rejected it: the API

server accepted the Pod and the scheduler placed it. Only the registry can say whether a digest exists.

- failed to resolve reference ... not found after docker.io/library/nginx@

tells you the repository was found and the digest was not. Compare with Lab 02, where the same phrasing appeared after nginx:1.27-alpin - there the **tag** was missing. Same error string, different last path segment; read it carefully.

- If instead you had seen 401 Unauthorized, the digest may well exist but your

node has no credentials. If you had seen no match for platform in manifest, you pinned a single-platform manifest digest and this node has a different architecture.

- Note what a stale digest cannot do: silently give you different content.

That is the entire trade - a digest fails loudly; a moved tag succeeds quietly with different bytes, which is what caused Meridian's outage.

Repair - pin to the digest this cluster resolved

```
CATALOG_ID=$(kubectl -n kcna-lab05 get pod image-catalog \
-o jsonpath='{.status.containerStatuses[0].imageID}')
echo "resolved digest: ${CATALOG_ID##*@}"
```

```
resolved digest: sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

image is one of the very few mutable Pod fields, so patch it in place:

```
kubectl -n kcna-lab05 set image pod/depot-web-locked "web=nginx@${CATALOG_ID##*@}"
```

```
pod/depot-web-locked image updated
```

```
kubectl -n kcna-lab05 wait --for=condition=Ready pod/depot-web-locked --timeout=120s
kubectl -n kcna-lab05 get pod depot-web-locked
```

```
pod/depot-web-locked condition met
NAME                READY   STATUS    RESTARTS   AGE
depot-web-locked    1/1     Running   0           2m11s
```

```
kubectl -n kcna-lab05 get pod depot-web-locked \
-o jsonpath='{.spec.containers[0].image}{"\n"}{.status.containerStatuses[0].imageID}{"\n"}'
```

```
nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

spec.image and status.imageID now name the same content. With the tag-pinned image-catalog Pod they do not, and cannot be made to. That single difference is the answer to the architecture board's third question, and it costs nothing at run time.

To make this practical rather than manual, Meridian's release pipeline should resolve the tag to a digest once at build time and commit the digest into the manifest:

```
# Illustrative - resolve once in CI, commit the result:
# crane digest nginx:1.27-alpine
# -> sha256:3d696e83...
# then render: image: nginx@sha256:3d696e83...
sed "s|nginx@sha256:[0-9a-f]\{64\}|nginx@${CATALOG_ID##*@}|" \
manifests/20-digest-pinned.yaml | grep '      image:'
```

```
image: nginx@sha256:3d696e8357051647b844d8c7cf4a0aa71e84379999a4f6af9b8ca1f7919ade42
```

Keep the human-readable tag in a label or annotation so the digest is still

traceable to a version, e.g.

app.kubernetes.io/version: "1.27-alpine".

7. Troubleshooting

Symptom	Likely cause	Fix
ErrImagePull ... failed to resolve reference "...@sha256:...": not found	The digest does not exist in that repository (stale, mistyped, or garbage-collected)	Re-resolve the digest from a known-good source and patch image; verify with status.imageID
ErrImagePull ... docker.io/library/x:tag: not found	The tag does not exist	Fix the tag (Lab 02); check for a typo or a retired release
ErrImageNeverPull	imagePullPolicy: Never and the image is not on the node	Pre-load it (kind load docker-image <ref>) or change the policy
ImagePullBackOff ... 401 Unauthorized / pull access denied	Private registry with no credentials	Create a kubernetes.io/dockerconfigjson Secret and add spec.imagePullSecrets (Lab 15)
no match for platform in manifest / exec format error on start	You pinned a single-platform manifest digest, or the image has no build for this node's architecture	Pin the index digest, or use a multi-arch tag; check <code>kubectl get node -o jsonpath='{.items[0].status.nodeInfo.architecture}'</code>
x509: certificate signed by unknown authority during pull	Corporate TLS-intercepting proxy, or a private registry with a private CA	Install the CA on the nodes; this cannot be fixed from a Pod spec
Two environments behave differently from the same manifest	A mutable tag resolved to different digests at different times	Compare status.imageID in both; move to digest pinning
Pod keeps re-pulling and start-up is slow	imagePullPolicy: Always (explicitly, or implied by :latest)	Use an explicit non-latest tag or a digest, and IfNotPresent
kubectl describe shows the image pulled but the container exits instantly	The image's ENTRYPOINT/CMD is not what you assumed	Revisit Lab 03; inspect the image config with a tool such as crane config <ref>

8. Cleanup

```
kubectl delete namespace kcna-lab05
```

```
namespace "kcna-lab05" deleted
```

9. What you learned

- An image reference is [REGISTRY/]REPOSITORY[:TAG][@DIGEST], and Kubernetes

applies two silent defaults: docker.io/library/ and :latest. latest means "the default tag", never "the newest".

- An OCI image is an index → manifest → config + content-addressed layers.

Layers are shared between images; the config is where ENTRYPOINT and CMD live.

- spec.image is a request; status.imageID is the answer. Record the

imageID at every release - it is the only durable statement of what ran.

- Tags are mutable pointers; digests are content addresses. A stale digest

fails loudly; a moved tag succeeds quietly with different bytes.

- Kubernetes talks to a runtime over the CRI; the runtime talks to runc

over the OCI runtime spec and to registries over the OCI distribution spec. Every Pod also runs a pause sandbox container that owns the network namespace.

- imagePullPolicy has three values and a default that depends on your tag.

ErrImageNeverPull is a policy outcome, not a network failure.

10. Further reading

- [Images](#) - reference format, imagePullPolicy, defaults and pull secrets
- [Container Runtime Interface \(CRI\)](#)
- [Container Runtimes](#)
- [Debugging Kubernetes nodes with crictl](#)
- [Don't Panic: Kubernetes and Docker \(dockershim removal\)](#)
- [OCI Image Specification](#)
- [OCI Distribution Specification](#)
- [OCI Runtime Specification](#)
- [Pod Lifecycle - container images and the sandbox](#)